Universidad de Alcalá

GUÍA DOCENTE

Metodología de la programación

Grado en
Matemáticas y Computación (GMyC)

Universidad de Alcalá

Curso Académico 2026/2027

1er Curso - 2º Cuatrimestre (GMyC)

Aprobada en Junta de Escuela de la EPS el día 15 de junio de 2026

GUÍA DOCENTE

Nombre de la asignatura:	**Metodología de la programación**
Código:	**592007 (GMyC)**
Titulación en la que se imparte:	**Grado en** **Matemáticas y Computación (GMyC)**
Departamento y Área de Conocimiento:	**Ciencias de la Computación** **Ciencias de la Computación**
Carácter:	**Básica (GMyC)**
Créditos ECTS:	**6.0**
Curso y cuatrimestre:	**1er Curso - 2º Cuatrimestre (GMyC)**
Profesorado:	Por definir
Horario de Tutoría:	Consultar al comienzo de la asignatura
Idioma en el que se imparte:	Español

1a. PRESENTACIÓN

La asignatura Programación pretende introducir a los alumnos en el paradigma de programación orientada a objetos. La asignatura presentará los conceptos teóricos asociados a este paradigma como clase, objeto, relaciones de herencia y asociación, clases abstractas y polimorfismo, etc., así como describir las técnicas de análisis y diseño orientado a objetos. Estos conocimientos teóricos se llevarán a la práctica mediante el lenguaje de programación orientado a objetos Java.

Prerrequisitos y Recomendaciones:

Es obligatorio haber cursado la asignatura de Fundamentos de Programación.

1b. COURSE SUMMARY

The Programming subject aims to introduce students to the object-oriented programming paradigm. The subject will present the theoretical concepts associated with this paradigm as a class, object, inheritance and association relationships, abstract classes and polymorphism, etc., as well as describe the techniques of object-oriented analysis and design. This theoretical knowledge will be implemented through the Java object-oriented programming language.

2. RESULTADOS DE APRENDIZAJE

Esta asignatura proporciona lo(s) siguiente(s) resultado(s) de aprendizaje:

C01 - Analizar, diseñar y construir aplicaciones robustas, seguras y eficientes, eligiendo el paradigma y los lenguajes de programación más adecuados.

K07 - Conocer las estrategias algorítmicas que puedan conducir a la resolución de un problema y los métodos para analizar su complejidad computacional.

3. CONTENIDOS

Bloques de contenido	Total de clases, créditos u horas
1. Introducción a los lenguajes de programación: Historia de los lenguajes de programación, breve presentación de los distintos paradigmas. Comparación entre intérpretes y compiladores; fases de la traducción de lenguajes; aspectos independientes y dependientes de la máquina. El concepto de máquina virtual, jerarquía de máquinas virtuales, lenguajes intermedios.	2 horas
2. Fundamentos de la programación orientada a objetos: Evolución hacia la programación orientada a objetos, crisis del software, calidad del software. Abstracción, tipos de datos, tipos abstractos de datos, diseño modular, reutilización. Diseño orientado a objetos, encapsulación y ocultación de información, separación entre comportamiento e implementación, clases, subclases y herencia, jerarquía de clases, clases abstractas, polimorfismo.	10 horas
3. Sintaxis del lenguaje de programación orientado a objetos: Revisión de tipos de datos, operadores, estructuras de control, control de acceso, clases, atributos, métodos.	2 horas
4. Análisis y diseño Orientado a Objetos: Identificación y modelado de clases y relaciones, lenguajes y notaciones de modelado, pruebas y documentación de programas.	10 horas
5. Tratamiento de errores: Mecanismos de tratamiento y recuperación de errores en tiempo de ejecución, excepciones, robustez de programas.	4 horas
6. Desarrollo de interfaces gráficas de usuario: Manejo de API, uso de API gráficas sencillas; diseño de GUI, programación dirigida por eventos, arquitectura MVC, programación reactiva, frameworks de diseño.	6 horas
7. Estructuras de datos y librería I/O: Formatos habituales: JSON, YAML, XML, EDI, CSV/TSV. Tratamiento de ficheros y técnicas de persistencia de objetos.	4 horas
8. Metodologías de la programación e Ingeniería del Software: Evolución. Revisión de distintas metodologías de desarrollo software y el proceso de Ingeniería del Software, desde la metodología en cascada, en espiral, metodologías ágiles hasta nuevas metodologías. Modelos de madurez.	2 horas
9. Avances en metodologías de desarrollo asistido con IA: Metodologías basadas en la automatización de operaciones. Monitorización y gestión del proceso de desarrollo con IA. Metodologías basadas en las especificaciones. Ciclo de vida con IA. Procesos automatizables.	16 horas

4. METODOLOGÍAS DE ENSEÑANZA APRENDIZAJE. ACTIVIDADES FORMATIVAS

4.1. Distribución de créditos (especificar en horas)

Número de horas presenciales:	60 horas (56 horas de clase presencial + 4 horas de evaluación)
Número de horas del trabajo propio del estudiante:	90 (Incluye horas de estudio, elaboración de actividades, preparación de exámenes)
Total horas	150

4.2. Estrategias metodológicas, materiales y recursos didácticos

Clases magistrales y expositivas, en combinación con prácticas en el laboratorio	Las clases magistrales están dedicadas a la difusión de conceptos básicos e imprescindibles para el correcto entendimiento de la materia, así como el establecimiento de una dinámica de trabajo adecuada entre los participantes del proceso de enseñanza-aprendizaje. Las sesiones de laboratorio estarán dedicadas a poner en práctica los conceptos adquiridos, permitiendo desarrollar al alumno las destrezas necesarias para su uso.
Trabajos en grupo y cooperativo	El trabajo cooperativo es de vital importancia en esta asignatura. Se potenciarán la programación mediante técnicas de pair-programming, alternando los alumnos en sus funciones entre sesiones. La realización de prácticas alternarán entre prácticas individuales y prácticas en grupo.
Trabajo y estudio personal	El trabajo personal en esta asignatura representa el pilar fundamental para poder superarla, ya que se necesitan por parte de los alumnos una dedicación constante para poder asimilar adecuadamente los conceptos impartidos. El trabajo personal se dividirá entre la construcción de prácticas a entregar y el estudio de la materia para las pruebas de evaluación teóricas.

5. EVALUACIÓN: Procedimientos, criterios de evaluación y calificación

Preferentemente se ofrecerá a los alumnos un sistema de evaluación continua que tenga características de evaluación formativa de manera que sirva de realimentación en el proceso de enseñanza-aprendizaje por parte del alumno.

5.1. PROCEDIMIENTOS

La evaluación debe estar inspirada en los criterios de evaluación continua (Normativa de Evaluación de los Aprendizajes, NEA, art 3). No obstante, respetando la normativa de la Universidad de Alcalá se pone a disposición del alumno un proceso alternativo de evaluación final de acuerdo a la Normativa de Evaluación de los Aprendizajes según lo indicado en su Artículo 10, los alumnos tendrán un plazo de quince días desde el inicio del curso para solicitar por escrito al Director de la Escuela Politécnica Superior su intención de acogerse al modelo de evaluación no continua aduciendo las razones que estimen convenientes. La evaluación del proceso de aprendizaje de todos los alumnos que no cursen solicitud al respecto o vean denegada la misma se realizará, por defecto, de acuerdo al modelo de evaluación continua. El estudiante dispone de dos convocatorias para superar la asignatura, una ordinaria y otra extraordinaria.

Convocatoria ordinaria

Evaluación continua:

Puesto que la materia de la asignatura tiene, principalmente, una utilidad práctica, la evaluación se centrará en el desarrollo y verificación de los aspectos prácticos incluyendo la aplicación de los conceptos estudiados y su verificación práctica.

Siguiendo esa línea, las principales herramientas de evaluación serán:

1. **Prueba de Laboratorio:** Desarrollo, entrega y defensa de una práctica completa.
2. **Entregables de Laboratorio (EL).** Realización de prácticas de laboratorio y entrega de las correspondientes memorias. La evaluación considerará la observación sistemática, donde el profesor registrará las principales dificultades y habilidades observadas en cada alumno, y la realización de una memoria única por práctica, por parte de cada uno de los grupos de alumnos que la hayan realizado.
3. **Prueba Teórica (PT).** Realización de pruebas escritas y/o test centradas en los aspectos relevantes de la asignatura.

Los alumnos deberán asistir al menos al 80% de las sesiones de laboratorio y entregar los informes correspondientes a todas las prácticas de laboratorio.

Los alumnos, en grupo, entregarán los informes de las prácticas de laboratorio siguiendo el calendario establecido. Estas prácticas serán evaluadas por el profesor responsable del grupo de laboratorio, para valorar si se han cumplido los objetivos indicados en el guion de la misma.

Evaluación mediante examen final:

En el caso de evaluación mediante examen final, los elementos de evaluación a emplear serán los siguientes:

1. **Prueba de laboratorio (PL).** Entrega de una práctica y defensa de la misma.

2. **Prueba Evaluación Final (PEF).** Similar a las pruebas de respuesta corta o de tipo test realizadas durante la evaluación continua.

Se recomienda a los alumnos que realicen las prácticas de laboratorio durante el desarrollo del cuatrimestre, sustituyendo de esta forma el examen práctico de laboratorio por la evaluación de las memorias correspondientes a las diferentes prácticas.

Convocatoria extraordinaria

El procedimiento será el mismo que el descrito para la evaluación mediante examen final en la convocatoria ordinaria.

5.2. EVALUACIÓN

CRITERIOS DE EVALUACIÓN

Se utilizarán los siguientes criterios para la evaluación de la asignatura, relacionados con los resultados del aprendizaje:

- CE1. El alumno conoce la historia y las características de los lenguajes de programación y es capaz de describir las diferencias entre los distintos paradigmas de programación.
- CE2. El alumno comprende la programación modular y conoce los criterios y principios del diseño modular.
- CE3. El alumno conoce los principios en los que se basa la orientación a objetos como abstracción, encapsulación, ocultamiento de la información, extensibilidad y reutilización.
- CE4. El alumno ha adquirido los conocimientos básicos del paradigma orientado a objetos tales como clase y objeto.
- CE5. El alumno ha adquirido los conocimientos avanzados del paradigma orientado a objetos tales como herencia, interface, clase abstracta y polimorfismo.
- CE6. El alumno sabe aplicar técnicas de análisis y diseño orientado a objetos expresarlo en un lenguaje de modelado como UML.
- CE7. El alumno sabe programar en un lenguaje orientado a objetos aplicando las técnicas del paradigma orientado a objetos y utilizar una herramienta de desarrollo.
- CE8. El alumno sabe desarrollar una aplicación orientada a objetos mediante una interfaz de usuario gráfica.
- CE9. El alumno es capaz de desarrollar una aplicación orientada a objetos robusta en la que se utilicen estructuras de datos complejas y persistencia
- CE10. El alumno es capaz de establecer y ejecutar satisfactoriamente un proceso de desarrollo de software asistido por IA.

INSTRUMENTOS DE EVALUACIÓN

Esta sección resume los instrumentos de calificación que serán aplicados a cada uno de los criterios de Evaluación.

- **Entregables de Laboratorio (EL):** Entregas de resultados y conclusiones de las prácticas guiadas propuestas a lo largo de la asignatura.
- **Prueba de Laboratorio (PL):** Práctica de laboratorio y defensa.
- **Prueba Teórica (PT):** Pruebas teóricas a realizar al finalizar la asignatura
- **Prueba de Evaluación Final (PEF):** Examen para aquellos alumnos que no realizan la evaluación continua en la convocatoria.

CRITERIOS DE CALIFICACIÓN

En la convocatoria **ordinaria–evaluación continua** la relación entre las competencias, resultados del aprendizaje, criterios e instrumentos de evaluación, es la siguiente.

Resultado de Aprendizaje	Criterio de Evaluación	Instrumento de evaluación	Peso en la calificación
C01, K07	CE2, CE3, CE4	EL1	10%
C01, K07	CE5, CE6, CE7, CE8, CE9	EL2	20%
C01, K07	CE1 - CE10	PL	40%
C01, K07	CE1 - CE6, CE8, CE9, CE10	PT	30%

Se otorgará la calificación de "No presentado" al alumno que habiendo optado por el procedimiento de evaluación continua, cumpla alguno de los siguientes requisitos:

- Cuando el alumno haya incumplido la asistencia al menos al 80% de las clases en laboratorios.
- Cuando el alumno no haya entregado TODOS los instrumentos de evaluación de la asignatura.

En la convocatoria **ordinaria–evaluación final** la relación entre las competencias, resultados del aprendizaje, criterios e instrumentos de evaluación, es la siguiente.

Resultado de Aprendizaje	Criterio de Evaluación	Instrumento de evaluación	Peso en la calificación
C01, K07	CE1 - CE10	PL	70%
C01, K07	CE1 - CE10	PEF	30%

Convocatoria extraordinaria

En el caso de la convocatoria extraordinaria se mantendrán los mismos porcentajes que se han establecido en el caso de la evaluación mediante examen final, pudiendo dar la opción de mantener las notas del conjunto completo de pruebas de laboratorio o pruebas teóricas, en caso de que estas tengan una media superior a 5 puntos sobre 10, y sólo realizar aquella prueba de la parte que quede por superar. El cualquier caso, el alumno que quiera acogerse a esta posibilidad deberá indicar este extremo al coordinador de la asignatura con una antelación de al menos 48 horas (2 días laborables) sobre la fecha oficial de evaluación (fecha de examen).

La metodología de enseñanza-aprendizaje y el proceso de evaluación se ajustarán cuando sea necesario, con las orientaciones de la Unidad de Atención a la Diversidad, para aplicar adaptaciones curriculares a los estudiantes con necesidades específicas.

6. BIBLIOGRAFÍA

6.1. Bibliografía básica

- Schutta, N., Vega, D. Fundamentals of Software Engineering. 1ª Edición, O'Reilly Media, 2025. (Accesible online https://learning.oreilly.com/library/view/fundamentals-of-software)
- Meyer, B. Construcción de Software Orientado a Objetos. 2ª Edición. Prentice Hall, 1998.
- Muñoz, C., Niño, A., Vizcaíno A. Introducción a la programación con orientación a objetos. Prentice Hall, 2003.
- Amescua, A. Análisis y diseño estructurado y orientado a objetos de sistemas informáticos. McGraw-Hill, 2003.
- Stevens, P., Pooley, R. Utilización de UML en ingeniería del software con objetos y componentes. Prentice Hall, 2002.

- Arnow, D., Weiss, G. Introducción a la programación con Java. Un enfoque orientado a objetos. Addison Wesley, 2001.
- Jiménez, A., Pérez, F. M. Aprende a programar con Java, 2ª edición. Paraninfo, 2016.
- Eckel, B. Piensa en Java. Cuarta Edición. Prentice Hall, 2007.
- Cadenhead, R. Programación Java 8. Anaya, 2014.
- Deitel, P., Deitel, H. Cómo programar en Java. 9ª edición. Pearson, 2012. 10ª edición actualizada a Java 8, 2015.

6.2. Bibliografía complementaria

- Martin R. Clean Code: A Handbook of Agile Software Craftsmanship. Pearson, 2009.
- Martin R. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Pearson, 2017.
- Perez, J. M. Problemas resueltos de programación en lenguaje Java. Thomson, 2003.
- Camacho, D. Programación, algoritmos y ejercicios resueltos en Java. Prentice Hall, 2003.
- Vozmediano, A.M. Java para novatos: Cómo aprender programación orientada a objetos con Java sin desesperarse en el intento. CreateSpace, 2017.
- Nadal, M. Curso de programación Java. Anaya, 2021.
- Joyanes, L. Programación orientada a objetos. Segunda Edición. McGraw-Hill, 1998.
- Martin, R. UML para programadores Java. Prentice Hall, 2004.

NOTA INFORMATIVA

Durante el desarrollo de las pruebas de evaluación han de seguirse las pautas marcadas en el Reglamento por el que se establecen las Normas de Convivencia de la Universidad de Alcalá, así como las posibles implicaciones de las irregularidades cometidas durante dichas pruebas, incluyendo las consecuencias por cometer fraude académico según el Reglamento de Régimen Disciplinario del Estudiantado de la Universidad de Alcalá.